

Programação Estruturada - A linguagem C (Parte 2)

Thyago Tenório Martins de Oliveira
tm.thyago@gmail.com

Penedo, 26 de Agosto de 2022
Núcleo de Excelência em Tecnologias Sociais - NEES
Universidade Federal de Alagoas – UFAL/ Pólo Penedo

Sumário

- 3 - Programação Estruturada - A linguagem C
- 3.4 - Operadores
 - 3.4.1 - Operadores aritméticos
 - 3.4.2 - Operadores lógicos e relacionais;
- 3.4.3 - Outros operadores: bit a bit, ?, &, *, sizeof, .
- 3.5 Expressões
 - 3.5.1 Ordem de avaliação
 - 3.5.2 Conversões de tipo e casts
 - 3.5.3 Espaçamento e parênteses

Programação Estruturada - A linguagem C

– Operadores Aritméticos (em detalhes)

Operador	Sintaxe
Adição unária	<code>+a</code>
Adição	<code>a + b</code>
Incremento pré-fixado	<code>++a</code>
Incremento pós-fixado	<code>a++</code>
Atribuição por adição	<code>a += b</code>
Subtração unária	<code>-a</code>
Subtração	<code>a - b</code>
Decremento pré-fixado	<code>--a</code>
Decremento pós-fixado	<code>a--</code>
Atribuição por subtração	<code>a -= b</code>

Programação Estruturada - A linguagem C

– Operadores Aritméticos (em detalhes) – Exemplos

```
int a = -1;  
int b = +a;  
int c = a + b;
```

```
int a = 1;  
int b = 2;  
b += a;  
printf("%d", b);
```

```
int a = 1;  
int b = ++a;  
printf("%d", b);
```

X

```
int a = 1;  
int b = a++;  
printf("%d", b);
```

Programação Estruturada - A linguagem C

– Operadores Aritméticos (em detalhes) – Exemplos

```
int a = -1;  
int b = -a;  
int c = a - b;  
printf("%d", c);
```

```
int a = 1;  
int b = 2;  
b -= a;  
printf("%d", b);
```

```
int a = 1;  
int b = --a;  
printf("%d", b);
```

X

```
int a = 1;  
int b = a--;  
printf("%d", b);
```

Programação Estruturada - A linguagem C

– Operadores Aritméticos (em detalhes)

Operador	Sintaxe
Multiplicação	$a * b$
Atribuição por multiplicação	$a *= b$
Divisão	a / b
Atribuição por divisão	$a /= b$
Módulo (resto)	$a \% b$
Atribuição por módulo (resto)	$a \% = b$

Programação Estruturada - A linguagem C

– Operadores Aritméticos (em detalhes) – Exemplos

```
int a = 1;  
int b = 2;  
int c = a*b;  
printf("%d", c);
```

```
int a = 1;  
int b = 2;  
b *= a;  
printf("%d", b);
```

```
int a = 9;  
int b = 2;  
int c = a/b;  
printf("%d", c);
```

```
int a = 9;  
int b = 2;  
a /= b;  
printf("%d", a);
```

```
int a = 9;  
int b = 2;  
int c = a % b;  
printf("%d", c);
```

```
int a = 9;  
int b = 2;  
a %= b;  
printf("%d", a);
```

Programação Estruturada - A linguagem C

– Operadores Comparativos (em detalhes)

Operador	Sintaxe
Menor que	<code>a < b</code>
Menor ou igual que	<code>a <= b</code>
Maior que	<code>a > b</code>
Maior ou igual que	<code>a >= b</code>
Diferente de	<code>a != b</code>
Igual a	<code>a == b</code>
Não lógico	<code>!a</code>
E lógico	<code>a && b</code>
Ou lógico	<code>a b</code>

Programação Estruturada - A linguagem C

– Operadores Comparativos (em detalhes)– Exemplos

```
int a = 5;  
int b = 3;  
int z = a < b;  
printf("%d", z);
```

```
int a = 5;  
int b = 3;  
int z = a <= b;  
printf("%d", z);
```

```
int a = 7;  
int b = 7;  
int z = a > b;  
printf("%d", z);
```

```
int a = 7;  
int b = 7;  
int z = a >= b;  
printf("%d", z);
```

```
int a = 7;  
int b = 7;  
int z = a != b;  
printf("%d", z);
```

```
int a = 7;  
int b = 7;  
int z = a == b;  
printf("%d", z);
```

Programação Estruturada - A linguagem C

– Operadores Comparativos (em detalhes)– Exemplos

```
int a = 7;  
int z = !a;  
printf("%d", z);
```

```
int a = 7;  
int b = 0;  
int z = a && b;  
printf("%d", z);
```

```
int a = 7;  
int b = 0;  
int z = a || b;  
printf("%d", z);
```

```
int a = 7;  
int b = 0;  
int z = !a && (a || b);  
printf("%d", z);
```

```
int a = 7;  
int b = 0;  
int z = (b && !b) || (b || a);  
printf("%d", z);
```

Programação Estruturada - A linguagem C

– Operadores Deslocamento de bits (em detalhes)

Operador	Sintaxe
Deslocamento à esquerda	<code>a << b</code>
Atribuição de deslocamento à esquerda	<code>a <<= b</code>
Deslocamento à direita	<code>a >> b</code>
Atribuição de deslocamento à direita	<code>a >>= b</code>

Programação Estruturada - A linguagem C

– Operadores Deslocamento de bits (em detalhes) –

Exemplos

```
int n1 = 14; //00001110
int n2 = n1 << 1; //00011100
printf("%d\n", n2);
```

```
int n1 = 17; //00010001
n1 = n1 >> 1; //00001000
printf("%d\n", n1);
```

```
int n1 = 17; //00010001
int n2 = 2;
n1 <<= n2; //01000100
printf("%d\n", n1);
```

```
int n1 = 17; //00010001
int n2 = 2;
n1 >>= n2; //00000100
printf("%d\n", n1);
```

Programação Estruturada - A linguagem C

– Operadores Lógicos de bits (em detalhes)

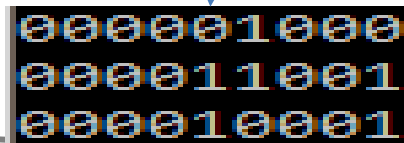
Operador	Sintaxe
Complemento	$\sim a$
E	$a \& b$
Atribuição por e	$a \&= b$
Ou	$a \mid b$
Atribuição por ou	$a \mid= b$
Ou exclusivo	$a \wedge b$
Atribuição por ou exclusivo	$a \wedge= b$

Programação Estruturada - A linguagem C

– Operadores Lógicos de bits (em detalhes) – Exemplos

```
int n1 = 8; //00001000
printf("%s\n", convert_binary(n1));
printf("%s\n", convert_binary(~n1));
```

```
int n1 = 8; //00001000
int n2 = 25; //00011001
printf("%s\n", convert_binary(n1 & n2));
printf("%s\n", convert_binary(n1 | n2));
printf("%s\n", convert_binary(n1 ^ n2));
```



000001000
000011001
000010001

Programação Estruturada - A linguagem C

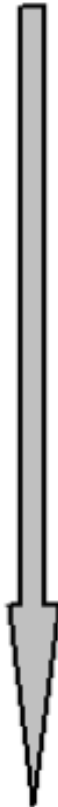
– Outros operadores (em detalhes)

Atribuição	<code>a = b</code>
Chamada de função	<code>a()</code>
Elemento de arranjo	<code>a[]</code>
De referência	<code>*a</code>
Referência	<code>&a</code>
Conversão de tipo de dados	<code>(tipo) a</code>
Vírgula	<code>a , b</code>
Tamanho de	<code>sizeof a</code>

Programação Estruturada - A linguagem C

- **Expressões**
- Precedência x Associatividade
 - **Precedência** refere-se a **ordem de operações**, por convenção, pela qual devem ser realizadas as operações numa expressão.
 - A **associatividade** de um operador é uma propriedade que determina como operadores de mesma precedência são agrupados na ausência de parêntesis.

Programação Estruturada - A linguagem C

GRUPOS DE OPERADORES	ASSOCIATIVIDADE	PRECEDÊNCIA
() , [] , -> , *	Esquerda para direita	Alta  Baixa
(<i>tipo</i>) [<i>conversão explícita</i>], sizeof, & [<i>endereço</i>], * [<i>indireção</i>], -, +, ~, ++, --, ! [<i>todos neste grupo são unários</i>]	Direita para Esquerda	
* [<i>multiplicação</i>], /, %	Esquerda para direita	
+, - [<i>binários</i>]	Esquerda para direita	
<<, >>	Esquerda para direita	
<, <=, >, >=	Esquerda para direita	
==, !=	Esquerda para direita	
& [<i>conjunção sobre bits</i>]	Esquerda para direita	
^	Esquerda para direita	
	Esquerda para direita	
&&	Esquerda para direita	
	Esquerda para direita	
? :	Direita para esquerda	
=, +=, -=, *=, /=, %=, <<=, >>=, &=, =, ^=	Direita para esquerda	
, [<i>vírgula</i>]	Esquerda para direita	

Obrigado!
Dúvidas, perguntas, sugestões?